Abstrakcja i enkapsulacja

Abstrakcja

Abstrakcja polega na skupieniu się na cechach istotnych dla rozważanego zagadnienia i pominięciu szczegółów, które nie są w danym momencie potrzebne. Tą samą nazwą określamy też powstałe w ten sposób pojęcie lub model. Nie chodzi więc o opisanie wszystkiego, lecz o wybranie tego, co jest ważne z określonego punktu widzenia. Takie użycie słowa odpowiada znaczeniom czynności i jej rezultatu w Wielkim słowniku języka polskiego PAN.

Przykładem jest pralka. Jej użytkownik musi wiedzieć, jak podłączyć wodę i prąd, gdzie włożyć pranie i wsypać proszek oraz jak wybrać program. Nie musi znać budowy pompy, termostatu, łożysk ani układów sterujących. Serwisant szukający przyczyny awarii schodzi na **niższy poziom abstrakcji**: interesują go te podzespoły i ich współdziałanie. Może jednak wymienić cały moduł sterowania bez naprawiania jego elektroniki. Producent tego modułu rozważa jeszcze inny poziom: używa elementów elektronicznych, których działanie zna, ale których wewnętrznej budowy nie musi sam projektować. Na każdym poziomie pewne szczegóły pomijamy, a inne stają się istotne.

Abstrakcja proceduralna — użycie procedur, funkcji, metod. Napisaną już funkcję/metodę (napisaną przez siebie lub np. z biblioteki standardowej) traktujemy jako zamkniętą jednostkę i interesuje nas tylko to jaki jest efekt jej działania, a nie sposób jej implementacji. Np. do użycia funkcji `math.sqrt` nie są potrzebne informacje o kolejnych krokach wykonywanych przez nią do obliczenia wyniku — istotna jest tylko informacja o tym, że metoda ta zwraca pierwiastek ze swojego argumentu.

W przypadku programowania obiektowego mówi się raczej o **abstrakcji danych**. Podobnie jak w przypadku abstrakcji proceduralnej oznacza ona to, że napisaną klasę (typ danych) traktujemy jako zamkniętą jednostkę. Do jej użycia nie musimy znać szczegółów implementacyjnych (np. wszystkich składowych klasy). Ważne jest tylko to jakie operacje możemy wykonać na obiektach klasy i jakie są efekty tych operacji.

Przykładem abstrakcji danych jest słownik, czyli typ `dict`. W programie korzystamy z operacji takich jak zapis i odczyt wartości pod kluczem, sprawdzanie obecności klucza czy usunięcie elementu. Nie musimy wiedzieć, jak pary klucz–wartość są przechowywane w pamięci. Wewnętrzna implementacja słowników w CPythonie była wielokrotnie zmieniana, np. w wersjach 3.3 i 3.6.

Programy korzystające z zachowanych reguł działania tych operacji nie musiały z tego powodu zmieniać sposobu używania słowników.

Abstrakcyjny typ danych (ang. *abstract data type*, ADT) — matematyczny model typu danych (klasy) — zbiór możliwych wartości oraz możliwych operacji na tych wartościach wraz z informacją jaki jest efekt działania tych operacji.

Umiejętność tworzenia odpowiednich abstrakcji jest ważna w wielu dziedzinach inżynierii, nie tylko w programowaniu. Pozwala zrozumieć istotne aspekty zagadnienia i na tej podstawie zaprojektować rozwiązanie, bez rozważania na każdym etapie wszystkich szczegółów. Dobór poziomu abstrakcji zależy od tego, co właśnie projektujemy lub próbujemy zrozumieć.

Enkapsulacja/Hermetyzacja

Hermetyzacja/**enkapsulacja** jest jedną z technik realizacji abstrakcji danych. Polega na określeniu podziału składowych klasy na publiczne i niepubliczne. Składowe publiczne są przeznaczone do używania przez kod korzystający z klasy. Składowe niepubliczne służą do realizacji jej działania: są istotne podczas pisania klasy, ale znajomość ich nie jest potrzebna do jej używania. To właśnie te „szczegóły”, które chcemy pominąć, gdy traktujemy klasę jako abstrakcję.

Często pola są składowymi niepublicznymi, a wybrane metody — publicznymi. Publiczne są metody realizujące operacje na obiektach istotne z punktu widzenia użytkownika klasy (programisty używającego klasy). Nie należą do nich metody pomocnicze, wykorzystywane jedynie do wykonania pewnych kroków w innych metodach.

Zestaw publicznych składowych wraz z informacją o tym, jak ich używać i jakie są efekty ich działania, nazywamy **publicznym interfejsem klasy**. Interfejs określa więc sposób korzystania z klasy, a nie to w jaki dokładnie sposób zostały zaimplementowane poszczególne operacje.

W analogii do pralki takim interfejsem są otwór na pranie, szufladka na proszek, przyłącza prądu i wody oraz panel sterowania. Podzespoły schowane w środku odpowiadają szczegółom implementacji. Słowo „szczegóły” nie oznacza, że są nieważne: bez nich pralka nie działałaby. Oznacza tylko, że wiedza o nich nie jest potrzebna do korzystania z urządzenia. Nowszy model może mieć zupełnie inne podzespoły, ale jeśli zachowuje te same funkcje i sposób ich wybierania, nie wymaga zmiany sposobu używania pralki.

W wielu językach programowania podział na składowe publiczne i prywatne jest egzekwowany przez mechanizmy języka, np. słowa kluczowe `public` i `private`. W Pythonie pojedyncze podkreślenie na początku nazwy, np. `_stan`,

jest konwencją oznaczającą składową niepubliczną. Wskazuje programiście, że nie powinien używać jej poza kodem realizującym działanie klasy, ale nie blokuje takiego dostępu technicznie. Pomaga unikać przypadkowego uzależnienia kodu od wewnętrznej implementacji.

Ochrona przed przypadkowym „zepsuciem" obiektu

Do wyjaśnienia przykładowych korzyści z enkapsulacji posłużymy się przykładową klasą `Konto` reprezentującą konto bankowe. Dla uproszczenia stan konta będzie po prostu liczbą całkowitą. Konto ma pewien dopuszczalny debet. Możemy sprawdzić stan konta, wolne środki (stan konta + dopuszczalny debet), wpłacić lub wypłacić pewną kwotę oraz zmodyfikować dopuszczalny debet. Na przykładzie klasy `Konto` pokazane zostanie jak enkapsulacja zabezpiecza przed omyłkowym „popsuciem" obiektu. Popsucie oznacza doprowadzenie do sytuacji, w której pola obiektu mają wartości, które nie reprezentują poprawnego konta, np. stan konta poniżej dopuszczalnego debetu lub dopuszczalny debet wyrażony jako napis a nie liczba.

Pierwsza wersja klasy `Konto` (brak enkapsulacji, część funkcjonalności dostępna za pomocą publicznych pól):

In [2]:
```
class Konto:
    def __init__(self, nr_konta):
        self.nr_konta = nr_konta
        self.stan = 0
        self.dopuszczalny_debet = 0

    def wolne_srodki(self):
        return self.stan + self.dopuszczalny_debet

    def wplata(self, kwota):
        self.stan += kwota

    def wyplata(self, kwota):
        if kwota > self.wolne_srodki():
            raise ValueError("brak środków")
        self.stan -= kwota
```

W klasie przyjęliśmy pewne założenia, które nie są na razie w żaden sposób jawnie wyrażone w kodzie źródłowym:

- pole `stan` powinno być typu `int`,
- pole `dopuszczalny_debet` również powinno być typu `int` i powinno przyjmować wartości nieujemne,
- do sprawdzenia i modyfikowania wartości dopuszczalnego debetu możemy używać bezpośrednio pola `dopuszczalny_debet`,

- do odczytu stanu konta możemy użyć pola `stan`, ale do jego modyfikacji powinniśmy używać metod `wplata` i `wyplata`.

Możemy utworzyć konto:

```
In [3]: k = Konto("12345")
```

```
In [4]: k.stan
```

```
Out[4]: 0
```

```
In [5]: k.dopuszczalny_debet
```

```
Out[5]: 0
```

```
In [6]: k.nr_konta
```

```
Out[6]: '12345'
```

Możemy wpłacić pieniądze na konto:

```
In [7]: k.wplata(1000)
```

```
In [8]: k.stan
```

```
Out[8]: 1000
```

Możemy zmienić dopuszczalny debet:

```
In [9]: k.dopuszczalny_debet = 1300
```

```
In [10]: k.wolne_srodki()
```

```
Out[10]: 2300
```

Możemy wypłacić pieniądze:

```
In [11]: k.wyplata(1500)
```

```
In [12]: k.stan
```

```
Out[12]: -500
```

```
In [13]: k.wolne_srodki()
```

```
Out[13]: 800
```

Nie możemy wypłacić więcej, niż wynoszą wolne środki. W poniższym przykładzie kwota `900` przekracza dostępne środki, więc metoda `wyplata` zgłosi wyjątek `ValueError`:

In [14]:
```
k.wyplata(900)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[14], line 1
----> 1 k.wyplata(900)

Cell In[2], line 15, in Konto.wyplata(self, kwota)
     13     def wyplata(self, kwota):
     14         if kwota > self.wolne_srodki():
---> 15             raise ValueError("brak środków")
     16         self.stan -= kwota

ValueError: brak środków
```

Niestety, niektóre niepoprawne operacje takie jak np. wypłata ujemnej kwoty nie zostaną wyłapane:

In [15]:
```
k.wyplata(-2000)
```

In [16]:
```
k.stan
```

Out[16]:
```
1500
```

In [17]:
```
k.wolne_srodki()
```

Out[17]:
```
2800
```

Nie powinniśmy modyfikować stanu konta inaczej niż przez wpłaty i wypłaty (czyli wywoływanie odpowiednich metod), ale nic nie chroni programu używającego klasy przed omyłkowym bezpośrednim zmniejszeniem pola `stan` zamiast wywołania metody `wyplata`:

In [18]:
```
k.stan -= 3000
```

Ominięty w ten sposób został mechanizm sprawdzający czy są wolne środki na wypłatę:

In [19]:
```
k.dopuszczalny_debet
```

Out[19]:
```
1300
```

In [20]:
```
k.stan
```

Out[20]:
```
-1500
```

Nic też nie chroni nas przed omyłkowym całkowicie błędnym użyciem klasy:

In [21]:
```
k.dopuszczalny_debet = "1300"
```

In [22]:
```
k.stan = "1000"
```

In [23]:
```
k.wolne_srodki()
```

Out[23]:
```
'10001300'
```

Błąd taki jak powyżej mógłby powstać np. gdyby w programie omyłkowo pominięto konwersję pobranych od użytkownika danych na liczby.

Spróbujemy dodać do klasy zabezpieczenia przed niektórymi omawianymi błędami. Dodajemy sprawdzenie czy wpłacana lub wypłacana kwota jest poprawna (poprawny typ i wartość) — metoda `sprawdz_kwote`.

Uwaga: preferowaną metodą sprawdzania poprawności typu jest w Pythonie funkcja `isinstance` a nie `type`. Działanie tej funkcji będzie omówione przy okazji tematu dziedziczenia.

Jako bonus dodajemy też zapisanie do pliku tekstowego informacji o operacjach wpłaty, wypłaty i sprawdzeniu wolnych środków (tworzenie tzw. *logu* — metoda `log`). **Uwaga:** w rzeczywistej aplikacji do tego celu powinno się raczej używać specjalnych bibliotek (np. moduł `logging` ze standardowej biblioteki Pythona).

Druga wersja klasy `Konto` (nadal brak enkapsulacji, dodana walidacja parametrów, dodane tworzenie logu):

In [24]:
```
class Konto:
    def __init__(self, nr_konta):
        self.nr_konta = nr_konta
        self.stan = 0
        self.dopuszczalny_debet = 0

    def sprawdz_kwote(self, kwota):
        if not isinstance(kwota, int):
            raise TypeError("kwota musi być typu int")
        if kwota <= 0:
            raise ValueError("kwota musi być dodatnia")

    def log(self, komunikat):
        with open(
            f"{self.nr_konta}_log.txt", "a", encoding="utf-8"
        ) as log:
            print(komunikat, file=log)
            print(f"stan konta po operacji: {self.stan}", file=log)
            print("---", file=log)

    def wolne_srodki(self):
        self.log("sprawdzenie wolnych środków")
        return self.stan + self.dopuszczalny_debet

    def wplata(self, kwota):
        self.sprawdz_kwote(kwota)
```

```
        self.stan += kwota
        self.log(f"wpłata: {kwota}")

    def wyplata(self, kwota):
        self.sprawdz_kwote(kwota)
        if kwota > self.wolne_srodki():
            raise ValueError("brak środków")
        self.stan -= kwota
        self.log(f"wypłata: {kwota}")
```

Wprowadzenie metod `sprawdz_kwote` i `log` dodaje kolejne założenie dotyczące używania klasy: są to metody pomocnicze, przeznaczone do wywoływania przez inne metody, a nie samodzielne operacje, które użytkownik klasy ma wykonywać na koncie. W obecnym kodzie ta różnica nie jest jeszcze widoczna w nazwach. Nic nie odróżnia ich pod tym względem od metod `wplata`, `wyplata` czy `wolne_srodki`.

Zobaczmy jak zachowuje się nowa wersja klasy.

In [25]:
```
k = Konto("12345")
```

Wypłacić można wyłącznie kwotę dodatnią. Próba wypłaty ujemnej kwoty zostanie teraz odrzucona przez metodę `sprawdz_kwote`, która zgłosi wyjątek `ValueError`:

In [26]:
```
k.wyplata(-3000)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[26], line 1
----> 1 k.wyplata(-3000)

Cell In[24], line 29, in Konto.wyplata(self, kwota)
     28     def wyplata(self, kwota):
---> 29         self.sprawdz_kwote(kwota)
     30         if kwota > self.wolne_srodki():
     31             raise ValueError("brak środków")
     32         self.stan -= kwota

Cell In[24], line 11, in Konto.sprawdz_kwote(self, kwota)
      7     def sprawdz_kwote(self, kwota):
      8         if not isinstance(kwota, int):
      9             raise TypeError("kwota musi być typu int")
     10         if kwota <= 0:
---> 11             raise ValueError("kwota musi być dodatnia")

ValueError: kwota musi być dodatnia
```

Podobnie metoda `wplata` wymaga dodatniej kwoty. Wpłata kwoty równej zero spowoduje wyjątek `ValueError`:

In [27]:
```
k.wplata(0)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[27], line 1
----> 1 k.wplata(0)

Cell In[24], line 24, in Konto.wplata(self, kwota)
     23     def wplata(self, kwota):
---> 24         self.sprawdz_kwote(kwota)
     25         self.stan += kwota
     26         self.log(f"wpłata: {kwota}")

Cell In[24], line 11, in Konto.sprawdz_kwote(self, kwota)
      7     def sprawdz_kwote(self, kwota):
      8         if not isinstance(kwota, int):
      9             raise TypeError("kwota musi być typu int")
     10         if kwota <= 0:
---> 11             raise ValueError("kwota musi być dodatnia")

ValueError: kwota musi być dodatnia
```

Niestety jeżeli jakaś operacja jest realizowana za pomocą odczytu lub zapisu pojedynczego pola (np. sprawdzenie stanu konta, sprawdzenie lub modyfikacja maksymalnego debetu), to nie mamy gdzie dodać dodatkowych operacji (zapis informacji do pliku, sprawdzenie poprawności przy modyfikacji wartości).

W dalszym ciągu nic nas nie chroni przed omyłkowym bezpośrednim użyciem pola `stan` ani przed podstawieniem błędnych wartości pod pola.

In [28]:
```
k.stan = -10000
```

W trzeciej wersji klasy `Konto` dokonujemy podziału na składowe niepubliczne (pola `_nr_konta`, `_stan` i `_dopuszczalny_debet` oraz metody `_sprawdz_kwote` i `_log`) oraz publiczne (pozostałe metody). Sprawdzenie stanu konta może być teraz wykonane przez metodę `stan`. Podobnie dopuszczalny debet można sprawdzić metodą `dopuszczalny_debet` i ustawić metodą `ustaw_dopuszczalny_debet`.

Trzecia wersja klasy `Konto` (enkapsulacja, cała istotna funkcjonalność jest zaimplementowana za pomocą metod):

In [29]:
```
class Konto:
    def __init__(self, nr_konta):
        self._nr_konta = nr_konta
        self._stan = 0
        self._dopuszczalny_debet = 0

    def _sprawdz_kwote(self, kwota):
        if not isinstance(kwota, int):
            raise TypeError("kwota musi być typu int")
        if kwota <= 0:
            raise ValueError("kwota musi być dodatnia")
```

```
    def _log(self, komunikat):
        with open(
            f"{self._nr_konta}_log.txt", "a", encoding="utf-8"
        ) as log:
            print(komunikat, file=log)
            print(f"stan konta po operacji: {self._stan}", file=log)
            print("---", file=log)

    def stan(self):
        self._log("sprawdzenie stanu")
        return self._stan

    def dopuszczalny_debet(self):
        self._log("sprawdzenie dopuszczalnego debetu")
        return self._dopuszczalny_debet

    def ustaw_dopuszczalny_debet(self, dopuszczalny_debet):
        if not isinstance(dopuszczalny_debet, int):
            raise TypeError("debet musi być typu int")
        if dopuszczalny_debet < 0 or self._stan < -dopuszczalny_debet:
            raise ValueError("niedopuszczalny debet")
        self._log(
            f"ustawienie dopuszczalnego debetu na {dopuszczalny_debet}"
        )
        self._dopuszczalny_debet = dopuszczalny_debet

    def wolne_srodki(self):
        self._log("sprawdzenie wolnych środków")
        return self._stan + self._dopuszczalny_debet

    def wplata(self, kwota):
        self._sprawdz_kwote(kwota)
        self._stan += kwota
        self._log(f"wpłata: {kwota}")

    def wyplata(self, kwota):
        self._sprawdz_kwote(kwota)
        if kwota > self.wolne_srodki():
            raise ValueError("brak środków")
        self._stan -= kwota
        self._log(f"wypłata: {kwota}")
```

Każda operacja istotna dla użytkownika konta jest teraz wykonywana za pomocą metody, więc każdą z nich możemy uwzględnić w logowaniu. Nie ma już publicznych pól, których odczyt lub zmiana omijałyby taki zapis.

Sprawdźmy tę wersję.

Jeżeli nie będziemy używać składowych niepublicznych, to nie ma teraz możliwości przypadkowego popełnienia błędów pokazanych poprzednio.

In [30]:
```
k = Konto("1234")
```

In [31]:
```
k.stan()
```

Out[31]:
```
0
```

In [32]:
```
k.ustaw_dopuszczalny_debet(1300)
```

In [33]:
```
k.wolne_srodki()
```

Out[33]:
```
1300
```

In [34]:
```
k.wyplata(700)
```

In [35]:
```
k.stan()
```

Out[35]:
```
-700
```

In [36]:
```
k.wolne_srodki()
```

Out[36]:
```
600
```

Metoda ustawiająca dopuszczalny debet sprawdza typ przekazanej wartości. Próba przekazania napisu zamiast liczby całkowitej spowoduje wyjątek `TypeError`:

In [37]:
```
k.ustaw_dopuszczalny_debet("700")
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[37], line 1
----> 1 k.ustaw_dopuszczalny_debet("700")

Cell In[29], line 29, in Konto.ustaw_dopuszczalny_debet(self, dopuszczalny_d
ebet)
     27     def ustaw_dopuszczalny_debet(self, dopuszczalny_debet):
     28         if not isinstance(dopuszczalny_debet, int):
---> 29             raise TypeError("debet musi być typu int")
     30         if dopuszczalny_debet < 0 or self._stan < -dopuszczalny_debe
t:
     31             raise ValueError("niedopuszczalny debet")
     32         self._log(f"ustawienie dopuszczalnego debetu na {dopuszczaln
y_debet}")

TypeError: debet musi być typu int
```

Nie można też wypłacić ujemnej kwoty. Poniższe wywołanie metody `wyplata` spowoduje wyjątek `ValueError`:

In [38]:
```
k.wyplata(-10000)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[38], line 1
----> 1 k.wyplata(-10000)

Cell In[29], line 45, in Konto.wyplata(self, kwota)
     44     def wyplata(self, kwota):
---> 45         self._sprawdz_kwote(kwota)
     46         if kwota > self.wolne_srodki():
     47             raise ValueError("brak środków")
     48         self._stan -= kwota

Cell In[29], line 11, in Konto._sprawdz_kwote(self, kwota)
      7     def _sprawdz_kwote(self, kwota):
      8         if not isinstance(kwota, int):
      9             raise TypeError("kwota musi być typu int")
     10         if kwota <= 0:
---> 11             raise ValueError("kwota musi być dodatnia")

ValueError: kwota musi być dodatnia
```

Widzimy, że powyższe operacje spowodowały wyjątki i nie wpłynęły na stan konta:

In [39]:
```
k.stan()
```

Out[39]:
```
-700
```